

Núcleo do BetterDev - plano, memória e hooks

Contexto

Implementação dos passos 1 a 3 da ordem de construção do [CONCEPCAO](#): formato de plano, memória por entradas e hooks que mantêm o sistema sozinho. O `atlas` está pausado, então a tese é medida no MDT durante a migração de customtkinter para pywebview, comparando com as 19 sessões já gravadas (25/08 a 21/09). Stack: Node/TypeScript; v1 só para Claude Code.

Fora do escopo

- Visualizador (`betterdev watch` , UI web, SQLite): plano próprio depois deste.
- `/refresh` , auditores (`/performance` , `/security`), `/identidade-visual` e `/stitch` .
- Hosts além do Claude Code.
- Publicação no npm e no marketplace: nesta versão o CLI é instalado com `npm link` e o plugin carregado com `claude --plugin-dir` .
- Telemetria de qualquer tipo. As métricas ficam em `.dev/.local/` , na máquina.

Restrições herdadas

Ainda não há entradas em `.dev/memory` neste repositório. Restrições que vêm da concepção e das referências:

- Nenhum estado mutável no plano: status sai de commits com o trailer `Plan-Step: <plano>/<fase>` ([CONCEPCAO §3.3](#)).
- Hooks rodam em forma `exec` (`"command": "node"` , `"args": [...]`): no Windows, os atalhos `.cmd` do npm não executam nesse modo (documentação oficial de hooks).
- O hook `Stop` tem limite de 8 continuações seguidas e recebe `stop_hook_active` ; ele deve usar `additionalContext` para dar retorno, não erro ([referencias/claude-code-best-practices.md](#)).
- Corpo de cada [SKILL.md](#) ≤ 8 KB, com o essencial no topo; receita positiva em vez de proibição ([CONCEPCAO §4.4](#)).
- Mecânica em script, julgamento no modelo da sessão ([CONCEPCAO §5 item 8](#)).
- Os históricos do Claude Code não marcam compactação de forma explícita: nas 19 sessões do MDT não há nenhum registro de compactação. A contagem histórica é estimada; a partir deste plano, o hook `PostCompact` registra.

f1 · Fundação, formato de plano e /plan

Objetivo

Criar o projeto TypeScript com o CLI `betterdev`, o parser e o validador do formato de plano, e o plugin com a skill `/plan`. Ao final, um plano gerado numa conversa real passa no `betterdev lint`.

Escopo

Código

- **Projeto:** `package.json` com `"type": "module"`, `engines.node >= 20`, `bin: { betterdev: dist/cli.js }` e scripts `typecheck ( tsc --noEmit )`, `test ( vitest run )` e `build ( esbuild gerando um único arquivo dist/cli.js , com dependências embutidas)`. Dependência de execução: só `yaml`. `tsconfig.json` com `strict`.
- `dist/` é **versionado**. O Claude Code não roda `build` ao instalar um plugin; os hooks e o `bin/` executam o `dist/cli.js` diretamente.
- `src/cli.ts` : roteador de subcomandos com `node:util` `parseArgs` , sem dependência extra. Nesta fase: `lint` e `init` .
- `src/plan/parse.ts` : lê o frontmatter com `yaml` e percorre o corpo linha a linha, reconhecendo títulos (`# - #####`) e **ignorando títulos dentro de blocos de código**. Devolve o plano estruturado: fases do YAML e, para cada `## fN · título` , as seções com linha inicial e final.
- `src/plan/lint.ts` : regras, cada uma com código e mensagem:
 - ids de fase únicos, no formato `f<número>` ; ids nunca precisam ser sequenciais;
 - cada fase do YAML tem um `## fN · título` no corpo com o **mesmo título**, e vice-versa;
 - `needs` só referencia ids existentes e não forma ciclo;
 - `files` e `verify` presentes e não vazios; `commit_msg` no formato `tipo: texto` , com `tipo` em `feat|fix|refactor|test|docs|chore|perf|style|build|ci` ;
 - seções obrigatórias: `## Contexto` e, em cada fase, `### Objetivo` , `### Escopo` > `##### Código` e `### Validação visual` ;
 - `### Objetivo` com no máximo 3 linhas de texto (aviso, não erro).
- `src/init.ts` (`betterdev init`): cria `.dev/{plans,memory,errors,templates,.local}` , copia `templates/plan.md` para `.dev/templates/` e acrescenta `.dev/.local/` ao `.gitignore` . É idempotente: não sobrescreve nada que exista e **nunca edita o** `CLAUDE.md` (isso fica com o `/setup` , na f5).
- **Plugin:**
 - `.claude-plugin/plugin.json` com nome `betterdev` , versão e descrição;
 - `bin/betterdev` : script sh de uma linha que executa `node "<pasta do script>/../dist/cli.js" "$@"` . O Claude Code põe `bin/` no PATH da ferramenta Bash, então dentro das sessões o comando funciona sem instalação global;
 - `skills/plan/SKILL.md` com `disable-model-invocation: true` . O conteúdo é o prompt de planejamento da §4.3 do CONCEPCAO, apontando para `${CLAUDE_PLUGIN_ROOT}/templates/plan.md` . Ordem do corpo: (1) roda nesta sessão, usando a conversa como contexto; (2) proporcionalidade: se o diff cabe numa frase, responde sem plano; (3) preenche o template; (4) grava em `.dev/plans/<id>.md` ; (5) roda `betterdev lint` e corrige até passar.

Interface

Saída do `betterdev lint` , uma linha por problema, com código de saída 1 quando há erro:

```
.dev/plans/e4.md:34 erro phase-title-mismatch "## f2 · Endpoint A/B" difere do YAML "Endpoint de diff A/B"
.dev/plans/e4.md:12 erro needs-unknown f3 depende de "f9", que não existe
.dev/plans/e4.md:41 aviso objective-too-long Objetivo da f2 tem 5 linhas (máximo 3)
2 erros, 1 aviso
```

Testes

- `tests/plan/parse.test.ts` : plano de uma fase; plano com três fases e `needs` ; título dentro de bloco de código ignorado; frontmatter ausente ou inválido gera erro claro.
- `tests/plan/lint.test.ts` : um fixture por regra em `tests/fixtures/plans/` (título divergente, fase sem corpo, corpo sem fase, `needs` desconhecido, ciclo, `commit_msg` fora do padrão, seção obrigatória ausente, objetivo longo). O próprio `templates/plan.md` preenchido com exemplo passa sem erro.
- `tests/init/init.test.ts` : em diretório temporário, cria a estrutura; segunda execução não altera nada; `CLAUDE.md` existente fica intacto.

Validação visual

- Rodar `npm link` e depois `betterdev lint .dev/plans/v1-nucleo.md` na raiz deste repositório: o plano passa sem erros.
- Estragar de propósito o título da f2 no corpo e rodar de novo: a mensagem aponta arquivo, linha e os dois títulos.
- Abrir `claude --plugin-dir .` num projeto de teste, discutir uma mudança pequena e rodar `/betterdev:plan` . Conferir que o arquivo foi gravado em `.dev/plans/` , passa no lint e lê bem como prosa.
- Na mesma sessão, pedir `/betterdev:plan` para algo trivial ("renomeie a variável x"): a skill responde sem gerar plano.
- Esta fase termina antes dos git hooks existirem. Fazer o commit com `git commit --trailer "Plan-Step: v1-nucleo/f1"` para que a f2 enxergue a f1 como concluída.

f2 · Status derivado do git e fase ativa

Objetivo

Derivar o status de cada fase dos commits, registrar localmente qual fase está ativa e preencher o commit com a mensagem e o trailer da fase. Ao final, `betterdev status` mostra o plano inteiro sem nenhum campo de status escrito no `.md` .

Escopo

Código

- `src/git/log.ts` : lê os trailers com `git log --format=%H%x1f%(trailers:key=Plan-Step,valueonly,separator=%x1e)%x1e` . Padrão: histórico de `HEAD` ; opção `--all` para todas as branches. Devolve um mapa `plano/fase → commits` .
- `src/plan/status.ts` : calcula o status de cada fase:
 - **concluída**: existe commit com o trailer;
 - **bloqueada**: alguma dependência não está concluída. A dependência é `needs` , ou a fase anterior quando `needs` está ausente;
 - **ativa**: é a fase gravada no estado local;
 - **pendente**: nenhum dos anteriores.
- `src/state.ts` : lê e grava `.dev/.local/state.json` (`{ plan, phase, startedAt }`). `betterdev start <plano>/<fase>` valida que a fase existe e avisa se está bloqueada (`--force` para ativar mesmo assim). `betterdev stop` limpa o estado.
- `src/plan/brief.ts` (`betterdev brief`): imprime só a fase ativa: a entrada dela no YAML e o bloco `## fn` do corpo, além de `Contexto` e `Fora do escopo` do plano. É o que a IA lê em vez do plano inteiro.
- `betterdev status [plano] [--json]` : tabela por fase; `--json` já no formato que o visualizador vai consumir.
- `src/git/hooks-install.ts` (`betterdev hooks install | uninstall`): instala `prepare-commit-msg` na pasta de hooks efetiva (respeita `core.hooksPath`):
 - arquivo ausente: cria;
 - arquivo com o marcador `# betterdev` : atualiza o bloco;
 - arquivo de outra ferramenta: acrescenta o bloco marcado no final, depois de salvar uma cópia `.bak` ;
 - `uninstall` remove só o bloco marcado.

O bloco chama `betterdev commit-msg "$1" "$2"` somente se o comando existir no PATH, e **nunca bloqueia um commit** (termina sempre com sucesso).

- `src/commit-msg.ts` (`betterdev commit-msg <arquivo> [origem]`): sem fase ativa, não faz nada. Em merge ou squash, não faz nada. Com mensagem vazia, preenche com o `commit_msg` da fase. Em todos os outros casos, acrescenta `Plan-Step: <plano>/<fase>` via `git interpret-trailers --if-exists doNothing` , sem duplicar.

Interface

```
$ betterdev status v1-nucleo
v1-nucleo · Núcleo do BetterDev - plano, memória e hooks
f1 concluída  Fundação, formato de plano e /plan          a3f2c91
f2 ativa     Status derivado do git e fase ativa
f3 pendente  Memória com frontmatter, índice gerado e /recall
f4 bloqueada Hooks de sessão, verificação no Stop e /wrap  aguarda f2, f3
f5 bloqueada Medição de sessões e /setup                 aguarda f4
```

Testes

- `tests/status/status.test.ts` : repositórios git temporários com commits reais. Casos: cadeia linear; `needs` paralelo (f3 liberada só com f1); fase bloqueada; fase ativa; trailer em outra branch visível só com `--all`.
- `tests/status/brief.test.ts` : o brief contém só a fase ativa, e o texto de outras fases não aparece.
- `tests/git-hooks/install.test.ts` : os três cenários de instalação, `core.hooksPath` configurado e `uninstall` preservando o hook alheio.
- `tests/git-hooks/commit-msg.test.ts` : mensagem vazia preenchida; mensagem existente recebe só o trailer; nenhum trailer duplicado em `--amend`; merge ignorado; sem fase ativa, arquivo intacto.

Validação visual

- Neste repositório: `betterdev hooks install`, depois `betterdev start v1-nucleo/f2` e `betterdev status`. A f1 aparece concluída (graças ao trailer manual) e a f2, ativa.
- `git commit --allow-empty` sem `-m`: o editor abre já com `feat: status de fases derivado do git` e o trailer.
- `betterdev brief` mostra só a f2, sem o texto das outras fases.
- `betterdev hooks uninstall` e conferir que `.git/hooks/prepare-commit-msg` voltou ao estado anterior.

f3 · Memória com frontmatter, índice gerado e /recall

Objetivo

Definir o formato das entradas de memória e de erros, validá-lo por script, gerar os índices automaticamente e buscar entradas pelo frontmatter. Ao final, o `/recall` encontra a entrada certa sem ler a memória inteira.

Escopo

Código

- `src/memory/schema.ts` : frontmatter das entradas.
 - Obrigatórios em ambas as trilhas: `track` (`conhecimento` | `bug`), `type` , `module` , `summary` (uma linha, ≤ 120 caracteres; vira a linha do índice), `tags` (1 a 8, minúsculas com hífen) e `created` .
Opcionais: `updated` e `applies_when` (até 5).
 - `type` é um enum fechado: `conhecimento` = `convencao` | `decisao` | `padrao` | `ferramenta` | `fluxo` | `pratica` ; `bug` = `build` | `teste` | `runtime` | `performance` | `dados` | `seguranca` | `ui` | `integracao` | `logica` .
 - Obrigatórios só na trilha `bug` : `symptoms` (1 a 5), `root_cause` , `resolution` e `occurrences` (começa em 1; o `/wrap` incrementa quando o mesmo erro volta).
- `src/memory/lint.ts` (entra no `betterdev lint`):
 - schema acima;
 - nome do arquivo igual ao slug, sem data;
 - corpo ≤ 40 linhas, `.dev/CLAUDE.md` ≤ 150 linhas, índice ≤ 200 linhas;
 - **regra do corpus**: aviso quando `module` ou uma tag é quase idêntica a um valor já usado.
Normaliza caixa, hífen e plural e considera distância de edição ≤ 2 , sugerindo o valor existente.
- `src/memory/reindex.ts` (`betterdev reindex`): gera `.dev/memory/index.md` e `.dev/errors/index.md` com o cabeçalho `<!-- GERADO por betterdev reindex; não editar -->` . Uma linha por entrada, ordenada por `module` e depois por slug. Saída determinística: mesma entrada, mesmos bytes.
 - Os índices ficam no `.gitignore` . São regenerados pelo hook de sessão (f4) e pelo `/wrap` , então nunca geram conflito de merge.
- `src/memory/entry.ts` (`betterdev entry new --track <t> --slug <s>`): cria o arquivo com o frontmatter da trilha pronto para preencher, e recusa se o slug já existir.
- `src/memory/recall.ts` (`betterdev recall <termos> [--full]`): pontua as entradas por coincidência dos termos, com peso decrescente em `tags` , `module` , `applies_when` , `summary` , título e `symptoms` . Lê só o frontmatter e devolve até 5 resultados com caminho, `summary` e campos que coincidiram. `--full` imprime também o corpo dos 3 primeiros.
- `skills/recall/SKILL.md` : roda `betterdev recall` , lê por completo só o que é relevante e **sinaliza quando uma entrada contradiz o código atual**, em vez de repeti-la. Pode ser invocada pelo modelo.

Interface

Linha do índice gerado:

```
- [viewshed-crs-metrico](viewshed-crs-metrico.md) · bug/dados · geom · crs, viewshed - Viewshed exige CRS métrico; em graus a máscara se
```

Saída do `betterdev recall viewshed crs` :

```
1. .dev/errors/viewshed-crs-metrico.md  bug/dados · geom
Viewshed exige CRS métrico; em graus a máscara sai deslocada
coincidiu: tags(crs, viewshed), module(geom)
```

Testes

- `tests/memory/schema.test.ts` : fixtures válidas nas duas trilhas; campo obrigatório ausente; `type` fora do enum; campos de bug exigidos só na trilha bug; tag com maiúscula.
- `tests/memory/corpus.test.ts` : `ui-web` VS `ui_web` , `teste` VS `testes` , `pywebview` VS `py-webview` geram aviso com sugestão; valores realmente distintos não geram.
- `tests/memory/reindex.test.ts` : saída idêntica em duas execuções; ordem estável; entrada nova aparece na linha certa; orçamento de 200 linhas acusado.
- `tests/memory/recall.test.ts` : a entrada com a tag exata vem primeiro; termo sem resultado devolve lista vazia sem erro; `--full` limita a 3 corpos.

Validação visual

- Num projeto de teste com `betterdev init` , criar 4 entradas com `betterdev entry new` (duas de cada trilha), rodar `betterdev reindex` e abrir os dois `index.md` : uma linha legível por entrada, sem precisar abrir os arquivos.
- Criar uma entrada com `module: Geom` e conferir o aviso sugerindo `geom` .
- Numa sessão com `claude --plugin-dir` , perguntar algo coberto por uma entrada e rodar `/betterdev:recall` . A resposta cita a entrada certa, e a skill lê por completo só ela.

f4 · Hooks de sessão, verificação no Stop e /wrap

Objetivo

Fechar o ciclo automático. Toda sessão começa com o estado do projeto injetado (também após compactação), o fim de cada turno roda a verificação da fase ativa quando os arquivos dela mudaram, e o `/wrap` registra no máximo um aprendizado, e só quando ele passa no critério de durabilidade.

Escopo

Código

- `hooks/hooks.json` : todos em forma `exec ( "command": "node", "args": ["${CLAUDE_PLUGIN_ROOT}/dist/cli.js", "hook", "<evento>"] )`:
 - `SessionStart` com matcher `startup|resume|clear|compact` ;
 - `Stop` com `timeout` de 300 s;
 - `PostCompact` .
- Todos os hooks são inertes fora de projetos BetterDev. Sem `.dev/` na raiz do projeto, terminam sem saída.
- `src/hooks/session-start.ts` : regenera os índices e emite `hookSpecificOutput.additionalContext` com:
 - branch e `git log --oneline -5` ;
 - `git diff --stat` resumido a 10 linhas;
 - fase ativa: plano, título, Objetivo, `files` e `verify` , com a indicação "detalhes: `betterdev brief` ";
 - os dois índices.

Teto de 8.000 caracteres: corta primeiro o diff e depois o índice mais longo, com marcador `[+N linhas: betterdev reindex]` . Cada injeção é registrada em `.dev/.local/metrics.jsonl` (`evento` , `source` , `caracteres` , `ts`).

- `src/hooks/stop.ts` : sai sem fazer nada se:
 - não há fase ativa;
 - `stop_hook_active` é verdadeiro;
 - `verifyOnStop` está desligado em `.dev/config.json` (padrão: ligado);
 - a impressão digital dos arquivos em `files` (hash do conteúdo) é igual à da última verificação aprovada.

Nos outros casos, roda os comandos de `verify` em sequência e grava a saída completa em `.dev/.local/verify/<data-hora>.log` :

- se todos passam: grava a impressão digital e sai em silêncio;
- se algum falha: responde com `hookSpecificOutput.additionalContext` contendo o comando que falhou, as **linhas de falha** (até 30, só as que parecem erro ou falha) e o caminho do log completo. O modelo continua e corrige, dentro do limite de 8 continuações do Claude Code.

Cada resultado vai para `metrics.jsonl` .

- `src/hooks/post-compact.ts` : registra `{ evento: "compact", trigger, session_id }` em `metrics.jsonl` . É a contagem de compactações daqui em diante.

- `src/config.ts` : lê `.dev/config.json` (opcional) com padrões seguros.
- `skills/wrap/SKILL.md` (`disable-model-invocation: true` , ≤ 8 KB). Receita, nesta ordem:
 1. Aplicar o **critério de durabilidade** ao que aconteceu na sessão: se a entrada sumisse, alguém lendo o código final repetiria o erro ou refaria uma investigação grande? Narrativa de sessão e lista de mudanças nunca passam. Se nada passa, responder "nada a registrar" com o motivo, e terminar.
 2. Rodar `betterdev recall` com os termos do aprendizado. Se o mesmo erro já existe, atualizar a entrada e incrementar `occurrences` , em vez de criar outra.
 3. Criar no máximo **uma** entrada com `betterdev entry new` e preenchê-la.
 4. Rodar `betterdev lint` e `betterdev reindex` até passar.
 5. Reportar em 2 a 3 linhas: o que foi registrado, onde, e por que passou no critério.

A decisão e a redação ficam no modelo da sessão; toda a validação é do script.

Interface

Contexto injetado no início da sessão (exemplo):

```
[betterdev] branch feat/v1-nucleo · fase ativa v1-nucleo/f4 - Hooks de sessão, verificação no Stop e /wrap
Objetivo: Fechar o ciclo automático...
Arquivos: hooks/hooks.json, src/hooks/*.ts, skills/wrap/SKILL.md
Verificação: npm run typecheck · npx vitest run tests/hooks · npm run build
Detalhes da fase: betterdev brief
Últimos commits: ...
Memória (3): ... Erros (2): ...
```

Retorno do `stop` quando a verificação falha:

```
[betterdev] verificação da fase f4 falhou: npx vitest run tests/hooks
FAIL tests/hooks/stop.test.ts > ignora quando stop_hook_active
AssertionError: expected 'block' to be undefined
Log completo: .dev/.local/verify/2026-09-23T14-02-11.log
```

Testes

- `tests/hooks/session-start.test.ts` : entrada JSON de exemplo via stdin produz o JSON esperado; projeto sem `.dev/` não produz saída; teto de 8.000 caracteres respeitado com os cortes na ordem certa; `source: compact` gera o mesmo contexto; caminhos com espaço.
- `tests/hooks/stop.test.ts` : sem fase ativa, nada; `stop_hook_active` verdadeiro, nada; impressão digital igual, não roda os comandos (verificado por comando espião); falha produz `additionalContext` com no máximo 30 linhas e o caminho do log; sucesso não produz saída e atualiza a impressão digital; `verifyOnStop: false` respeitado.
- `tests/hooks/post-compact.test.ts` : o evento é anexado a `metrics.jsonl` .
- `tests/hooks/wrap-skill.test.ts` : o `SKILL.md` tem ≤ 8 KB e contém, nesta ordem, o critério de durabilidade e o limite de uma entrada. É uma checagem estrutural, não do comportamento do modelo; esse é validado visualmente.

Validação visual

- `claude --plugin-dir .` neste repositório com a f4 ativa: rodar `/context` e conferir que o contexto injetado pelo hook aparece e fica abaixo de ~2k tokens.
- Quebrar um teste de `tests/hooks` e pedir ao Claude uma alteração qualquer em `src/hooks/`. No fim do turno, aparece o “Stop hook feedback” com a falha, e o Claude corrige sem você pedir.
- Terminar um turno sem mexer nos arquivos da fase: nada roda (conferir no `metrics.jsonl` que não há nova verificação).
- Rodar `/compact` e perguntar “qual é a fase ativa e o que falta nela?”: a resposta vem do contexto reinjetado, sem o Claude ler o plano.
- Rodar `/betterdev:wrap` numa sessão rotineira: a resposta deve ser “nada a registrar”. Numa sessão em que um problema não óbvio foi resolvido, deve surgir uma única entrada que passa no lint.

f5 · Medição de sessões e /setup para projetos existentes

Objetivo

Medir o antes e o depois a partir dos históricos que o Claude Code já grava, e migrar um projeto existente para o BetterDev. Ao final, o MDT está migrado, a linha de base das 19 sessões está calculada, e a comparação roda sozinha conforme você trabalha na migração para pywebview.

Escopo

Código

- `src/stats/transcripts.ts` : localiza os históricos do projeto em `~/.claude/projects/<slug>/`, onde o slug é o caminho com `:`, `\` e `/` trocados por `-` (ex.: `c--Users-awx1530897-Desktop-Projetos-MDT`). Se não encontrar, procura pastas cujos registros têm `cwd` igual ao projeto. Por sessão, ignorando registros de subagente (`isSidechain`):
 - **tokens de abertura:** `input_tokens + cache_read_input_tokens + cache_creation_input_tokens` da primeira resposta;
 - **tool calls até a primeira edição:** número de `tool_use` antes do primeiro `Edit`, `Write` ou `NotebookEdit`;
 - total de tool calls e duração;
 - **compactações:** do `metrics.jsonl` quando existir; senão, estimadas por queda de mais de 50% no contexto entre turnos seguidos, sempre marcadas como “estimado”.
- `src/stats/report.ts` (`betterdev stats [--project <caminho>] [--split-at <data>] [--json]`): tabela por sessão e medianas antes e depois da data de corte. Inclui as **ocorrências repetidas** de erros já registrados (soma de `occurrences - 1` na trilha bug) como sinal de qualidade.
- `src/migrate/split.ts` (`betterdev migrate split <arquivos...>`): parte mecânica da migração. Divide `MEMORY.md` e `ERRORS.md` pelos títulos, respeitando blocos de código, em candidatos em `.dev/.local/migration/NNN.md`, cada um com origem (arquivo e intervalo de linhas). Não decide nada.
- `templates/CLAUDE.md` : modelo do `.dev/CLAUDE.md` com:
 - a linha de descoberta **informativa** da §4.1 do CONCEPCAO;
 - as regras de mudança cirúrgica e de verificação (de `referencias/karpathy.md`);
 - instrução sobre o que preservar ao compactar (fase ativa, arquivos modificados, comandos de teste);
 - espaço para o conteúdo do projeto.
- `skills/setup/SKILL.md` (`disable-model-invocation: true`). Receita:
 1. `betterdev stats` para registrar a linha de base antes de qualquer mudança;
 2. `betterdev init`;
 3. `betterdev migrate split` nos arquivos de memória existentes;
 4. para cada candidato, em lotes, aplicar o critério de durabilidade: **descartar**, **fundir** com outro, ou **criar entrada** via `betterdev entry new`. Registrar cada decisão em `.dev/.local/migration/decisoes.md` (candidato, decisão, motivo);
 5. montar `.dev/CLAUDE.md` a partir do template e do `CLAUDE.md` atual, **removendo as instruções de “leia `MEMORY.md/ERRORS.md`”**;
 6. **pedir sua confirmação** antes de trocar o `CLAUDE.md` da raiz pelo atalho `@.dev/CLAUDE.md`;
 7. `git rm` de `MEMORY.md` e `ERRORS.md` num commit próprio; o conteúdo continua no histórico;
 8. `betterdev lint`, `betterdev reindex` e `betterdev hooks install`;
 9. relatório: quantos candidatos, quantos viraram entrada, quantos foram descartados ou fundidos, e o tamanho do contexto de abertura antes e depois.

Interface

```
$ betterdev stats --project ../MDT --split-at 2026-09-24
              antes (19 sessões)  depois (5 sessões)
tokens na abertura  38.412 (mediana)  3.180
tool calls até editar  14              5
compactações/sessão  0,4 (estimado)  0,2
erros repetidos      -              0
```

Testes

- `tests/stats/transcripts.test.ts` : históricos sintéticos em `tests/fixtures/transcripts/` (JSONL com os mesmos campos dos reais, sem conteúdo de conversa). Casos: tokens de abertura; contagem até a primeira edição; registros de subagente ignorados; sessão sem edição; compactação estimada por queda de contexto; slug de caminho Windows.
- `tests/stats/report.test.ts` : medianas e divisão por data; `--json` estável.
- `tests/migrate/split.test.ts` : títulos aninhados; título dentro de bloco de código não divide; intervalos de linha corretos; arquivo sem títulos vira um candidato só.
- `tests/migrate/setup-skill.test.ts` : checagem estrutural do [SKILL.md](#) (≤ 8 KB; os passos 1 e 6 presentes).
- `npm test` completo passa.

Validação visual

- Antes de migrar: `betterdev stats --project C:/Users/awx1530897/Desktop/Projetos/MDT` . Conferir que aparecem 19 sessões e que os tokens de abertura estão na ordem dos ~38k esperados pelo tamanho dos arquivos. Guardar a saída.
- No MDT, numa branch `chore/betterdev-setup` , rodar `/betterdev:setup` . Ler `decisoes.md` e conferir amostras: o que foi descartado era mesmo narrativa ou coisa recuperável do código? Aprovar a troca do `CLAUDE.md` .
- Abrir uma sessão nova no MDT e rodar `/context` : a parte de memória e instruções deve cair de ~38k para ≤ 4 k tokens.
- Fazer a primeira fase real da migração para pywebview com `/betterdev:plan` , `betterdev start` e o ciclo normal. Após ~5 sessões, rodar `betterdev stats --split-at <data da migração>` e comparar com os critérios combinados: abertura $\geq 10\times$ menor, menos tool calls até a primeira edição, compactações iguais ou menores, e nenhum erro já registrado se repetindo.
- Critério de qualidade manual: nas mesmas sessões, anotar se você precisou corrigir o Claude sobre algo que estava na memória antiga. Se isso acontecer mais do que antes, algum descarte da migração foi errado; revisar `decisoes.md` .